

RENANCE

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

CSC 318 — Past Questions & Materials
by Renance

Course: CSC 318
Institution: Federal University of Petroleum Resources, Effurun
Papers in this pack: 3
Levels: 300

This study pack was compiled and restructured by Renance for CSC 318 students at Federal University of Petroleum Resources, Effurun. It brings together 3 past paper(s)/material(s) into one clean, readable document so you can revise without chasing scattered PDFs. Practise these same questions interactively on Renance with instant answers, AI explanations, and full CBT mock exams — over 235,000+ questions across JAMB, Post-UTME, WAEC, NECO and university courses.

Study online: renance.app
Get the app: Google Play — [dev.renance.renance](https://play.google.com/store/apps/details?id=dev.renance.renance)

Generated by Renance AI

Paper 1 of 3 — CSC 318 Survey of Programming Languages 2024/2025.

Level 300 • First Semester • PastQuestion

This source paper is a scanned document; its pages are reproduced below.



FEDERAL UNIVERSITY OF PETROLEUM RESOURCES EFFURUN
DEPARTMENT OF COMPUTER SCIENCE
1ST SEMESTER UNDERGRADUATE EXAMINATION (2025/2024 SESSION)
CSC 318 – SURVEY OF PROGRAMMING LANGUAGES

INSTRUCTION: ANSWER QUESTION 1 AND ANY OTHER 3

TIME: 2 HOURS: 30 MINS

QUESTION 1 (25mks)

- a) State the history of any ten programming languages ✓
- b) Give an overview of high- and low-level language ✓
- c) Explain data binding ✓
- d) State and explain data binding types ✓
- e) With the aid of a well labelled diagram, explain how data binding works ✓

QUESTION 2 (15mks)

- a) Give an overview of programming paradigms. ✓
- b) Script programming as a computer language, explain stating the types, applications, advantages and disadvantages ✓
- c) State and explain the examples of data binding ✓
- d) State and explain the data binding tools ✓

QUESTION 3 (15mks)

- a) Give an overview of formal semantics ✓
- b) State the fields of formal semantics ✓
- c) Explain the approaches to formal semantics ✓
- d) Explain the variation of formal semantics. ✓
- e) i). Explain the concept of garbage collection under the following headings
ii). Give the advantages and disadvantages of garbage collection

QUESTION 4 (15mks)

- a) Give an overview of type checking ✓
- b) With examples, state and explain the kinds of type checking ✓
- c) state the benefits of each of the type checking ✓
- e) Explain declaration and data types ✓
- f) state and explain the different data types we have ✓

QUESTION 5 (15mks)

- a) Explain the concept of data visibility under the following headings:
 - i) The importance of data visibility
 - ii) Data visibility tools
 - iii) Data scope
 - iv) Data lifetime
- b) list and explain the object-oriented programming paradigms

EXAMINER: DR. NWOZOR B.U.

Paper 2 of 3 — CSC 318 PAST QUESTION

Level 300 • First Semester • PastQuestion

a FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

COLLEGE OF SCIENCE

DEPARTMENT OF COMPUTER SCIENCE

FIRST SEMESTER EXAMINATION 2022/2023 SESSION

CSC 318: Survey of Programming Languages Time: 2M hours

Instructions: Answer four questions s.

(Use a fresh page of your answer script for a new question and write the question number at of every page. Don't mix up answers).

Question 1

Briefly discuss the evolution of the following groups of programming languages | history of programming languages:

- a. Machine codes and assemblers. (4 marks)
- b. Lisp. SNOBOL, APL, BASIC. (4.5 marks)
- c. Algol, C, Pascal, PL/I Ada, Modula-2, C++ (4.5 marks)
- d. SmallTalk, Prolog; Icon, Perl (4.5 marks)

Question 2

- a. What do you understand about the following the following | Imperative programming. (4 marks)
- ii. Functional and object oriented programming. (4 marks)
- iii. Declarative programming. (4 marks)
- b. Explain pure versus impure or multi paradigms in (5.5 marks) :

Question 3

- a How will you classify Lisp as
- b. Given the list: (chair table f.
 - (i) What is the car
 - (ii) What is the cç

Question 4

- a. Describe context free grammar with an example. (7
 - b. Explain the difference between context free relationship with a diagram. (4.5 marks)
- Describe lexical analysis. (3 marks)
- |. Describe syntax analysis. (3 marks)

Question 5

- a. Given the production rule:
and the input string:

Paper 3 of 3 — CSC 318: SURVEY OF PROGRAMMING LANGUAGES (3 UNITS)

Level 300 • First Semester • Handout

CSC 318: SURVEY OF PROGRAMMING LANGUAGES (3 UNITS)

Overview of programming languages: History of programming languages, brief survey of programming paradigms (procedural languages, object oriented languages, functional languages, declarative and non-algorithmic languages, scripting languages), the effects of scale on programming methodology; language description: syntactic structure (Expression notations, abstract syntax tree, lexical syntax, grammars for expressions, variants of grammars), language semantics, (informal semantics, overview of formal semantics, denotation semantics, axiomatic semantic, operational semantics); Declarations and types: the concept of types, declaration models (binding, visibility, scope and lifetime), overview of type- checking, garbage collection, abstraction mechanisms: procedures, function and iterations as abstraction mechanisms, parametrization mechanisms (reference vs value),activation records and storage management, type programming languages; object oriented language paradigm; functional and logic language paradigms.

OVERVIEW OF PROGRAMMING LANGUAGES

A programming language is a set of instruction that allows a programmer to communicate with a computer in order to perform specific tasks. Programming languages are used to create apps, websites, and operating systems.

There have been many programming languages some of them are listed below:

C Python C++
 C# R Ruby
 COBOL ADA Java
 Fortran BASIC Altair BASIC
 True BASIC Visual BASIC
 GW BASIC
 QBASIC PureBASIC PASCAL
 Turbo Pascal GO ALGOL
 LISP SCALA Swift
 Rust Prolog
 Racket Scheme Simula
 Perl PHP Java Script
 Coffee Script Visual FoxPro Babel
 Logo
 Lua
 Smalltalk
 Matlab F F#

Dart Datalog dbase

Haskell dylan Julia

ksh metroMumps

Nim OCaml pick

TCL D CPL

Curry ActionScript Erlang

Clojure DarkBASIC Assembly

Most Popular Programming Languages –

C

Python

C++

Java

SCALA

C#

R

Ruby

Go

Swift

JavaScript

Characteristics of a programming Language –

A programming language must be simple, easy to learn and use, have good readability, and be human recognizable.

Abstraction is a must-have Characteristics for a programming language in which the ability to define the complex structure and then its degree of usability comes.

A portable programming language is always preferred.

Programming language's efficiency must be high so that it can be easily converted into a machine code and its execution consumes little space in memory.

A programming language should be well structured and documented so that it is suitable for application development.

Necessary tools for the development, debugging, testing, maintenance of a program must be provided by a programming language.

A programming language should provide a single environment known as Integrated Development Environment(IDE).

A programming language must be consistent in terms of syntax and semantics.

Basic Terminologies in Programming Languages:

Algorithm: A step-by-step procedure for solving a problem or performing a task.

Variable: A named storage location in memory that holds a value or data.

Data Type: A classification that specifies what type of data a variable can hold, such as integer, string, or Boolean.

Function: A self-contained block of code that performs a specific task and can be

called from other parts of the program.

Control Flow: The order in which statements are executed in a program, including loops and conditional statements.

Syntax: The set of rules that govern the structure and format of a programming language.

Comment: A piece of text in a program that is ignored by the compiler or interpreter, used to add notes or explanations to the code.

Debugging: The process of finding and fixing errors or bugs in a program.

IDE: Integrated Development Environment, a software application that provides a comprehensive development environment for coding, debugging, and testing.

Operator: A symbol or keyword that represents an action or operation to be performed on one or more values or variables, such as + (addition), - (subtraction), * (multiplication), and / (division).

Statement: A single line or instruction in a program that performs a specific action or operation.

Advantages of programming languages:

1. **Increased Productivity:** Programming languages provide a set of abstractions that allow developers to write code more quickly and efficiently.
2. **Portability:** Programs written in a high-level programming language can run on many different operating systems and platforms.
3. **Readability:** Well-designed programming languages can make code more readable and easier to understand for both the original author and other developers.
4. **Large Community:** Many programming languages have large communities of users and developers, which can provide support, libraries, and tools.

Disadvantages of programming languages:

1. **Complexity:** Some programming languages can be complex and difficult to learn, especially for beginners.
2. **Performance:** Programs written in high-level programming languages can run slower than programs written in lower-level languages.
3. **Limited Functionality:** Some programming languages may not have built-in support for certain types of tasks or may require additional libraries to perform certain functions.
4. **Fragmentation:** There are many different programming languages, which can lead to fragmentation and make it difficult to share code and collaborate with other developers.

HISTORY OF PROGRAMMING LANGUAGES

1843: Ada Lovelace's machine algorithm

Ada Lovelace invents the first-ever machine algorithm for Charles Babbage's

Difference Machine that lays the foundation for all programming languages.

1949: Assembly Language

Assembly language was used in the Electronic Delay Storage Automatic Calculator (EDSAC). Assembly language was a type of low-level programming language that simplified the language of machine code. In other words, the specific instructions necessary to operate a computer.

1949: Shortcode

Shortcode (or Short-order code), was the first High-Level Language (HLL) suggested by John McCauley in 1949. However, it was William Schmitt who implemented it for the BINAC computer the same year and for the UNIVAC in 1950.

1952: Autocode

Autocode was a general term used for a family of programming languages. First developed by Alick Glennie for the Mark 1 computer at the University of Manchester, Autocode was the first-ever compiled language to be implemented meaning that it can be translated directly into machine code using a program called a compiler. Autocode was used on the Ferranti Pegasus and Sirius early computing machines in addition to the Mark 1.

1957: FORTRAN

FORmula TRANslation or FORTRAN was created by John Backus and is considered to be the oldest programming language in use today. The programming language was created for high-level scientific, mathematical, and statistical computations. FORTRAN is still in use today in some of the world's most advanced supercomputers.

1958: ALGOL (Algorithmic Language)

Algorithmic language or ALGOL was created by a joint committee of American and European computer scientists. ALGOL served as the starting point for the development of some of the most important programming languages including Pascal, C, C++, and Java.

1958: LISP (List Processor)

List processor or LISP was invented by John McCarthy at the Massachusetts Institute of Technology (MIT). Originally purposed for artificial intelligence, LISP is one of the oldest programming languages still in use today and can be used in the place of Ruby or Python. Companies such as Acceleration, Boeing, and Genworks are still using LISP in their tech stacks.

1959: COBOL (Common Business Oriented Language)

Common Business Oriented Language (COBOL), is the programming language behind many credit card processors, ATMs, telephone and cell calls, hospital signals, and traffic signals systems (just to name a few). The development of the language was led by Dr. Grace Murray Hopper and was designed so that it could

run on all brands and types of computers. COBOL is still used to this day primarily for banking and gamification systems.

1964: BASIC (Beginner's All-Purpose Symbolic Instruction Code)

Beginners All-Purpose Symbolic Instruction Code or BASIC was developed by a group of students at Dartmouth College. The language was written for students who did not have a strong understanding of mathematics or computers. The language was developed further by Microsoft founders Bill Gates and Paul Allen and became the first marketable product of the company.

1970: PASCAL

Named after the French mathematician Blaise Pascal, Niklaus Wirth developed the programming language in his honor. It was developed as a learning tool for computer programming which meant it was easy to learn. It was favored by Apple in the company's early days, because of its ease of use and power.

1972: Smalltalk

Developed at the Xerox Palo Alto Research Centre by Alan Kay, Adele Goldberg, and Dan Ingalls, Smalltalk allowed for computer programmers to modify code on the fly. It introduced a variety of programming language aspects that are visible languages of today such as Python, Java, and Ruby. Companies such as Leafly, Logitech, and CrowdStrike state they use Smalltalk in their tech stacks.

1972: C

Developed by Dennis Ritchie at the Bell Telephone Laboratories for use with the Unix operating system. It was called C because it was based on an earlier language called 'B'. Many of the current leading languages are derivatives of C including; C#, Java, JavaScript, Perl, PHP, and Python. It also has been/still being used by huge companies like Google, Facebook, and Apple.

1980/81: Ada

Ada was originally designed by a team led by Jean Ichbiah of CUU Honeywell Bull under contract to the United States Department of Defense. Named after the mid-19th-century mathematician Ada Lovelace, Ada is a structured, statically typed, imperative, wide-spectrum, and object-oriented high-level programming language. Ada was extended from other popular programming languages at the time such as Pascal. Ada is used for air-traffic management systems in countries such as Australia, Belgium, and Germany as well as a host of other transport and space projects.

1983: C++

Bjarne Stroustrup modified the C language at the Bell Labs, C++ is an extension of C with enhancements such as classes, virtual functions, and templates. It has been listed in the top 10 programming languages since 1986 and received Hall of Fame status in 2003. C++ is used in MS Office, Adobe Photoshop, game engines, and other high-performance software.

1987: Perl

Perl was created by Larry Wall and is a general-purpose, high-level programming language. It was originally designed as a scripting language designed for text editing but nowadays it's widely used for many purposes such as CGI, database applications, system administration, network programming, and graphic programming.

1990: Haskell

Haskell is a general-purpose programming language named after the American logician and mathematician Haskell Brooks Curry. It is a purely functional programming language meaning it's primarily mathematical. It's used across multiple industries particularly those that deal with complicated calculations, records, and number-crunching. Like many other programming languages from this era, it is not overly common to see Haskell in use for well-known applications. With that said, the programming language has been used to write a number of games one of which is Nikki and the Robots.

1991: Python

Named after the British comedy troupe 'Monty Python', Python was developed by Guido Van Rossum. It is a general-purpose, high-level programming language created to support a variety of programming styles and be fun to use (a number of the tutorials, samples, and instructions often contain Monty Python references). Python is, to this day, one of the most popular programming languages in the world is used by companies such as Google, yahoo, and Spotify.

1991: Visual Basic

Developed by Microsoft, Visual Basic allows programmers to utilize a drag-and-drop style of choosing and changing pre-selected chunks of code through a graphical user interface (GUI). The language is not overly used these days however Microsoft has used portions Visual Basic to a number of their applications like Word, Excel, and Access.

1993: Ruby

Created by Yukihiro Matsumoto, Ruby is an interpreted high-level programming language. A teaching language which was influenced by Perl, Ada, Lisp, and Smalltalk – among others. Ruby's primary uses are for web applications development and Ruby on Rails. Twitter, Hulu, and Groupon are some well-known examples of companies that use Ruby.

1995: Java

Java is a general-purpose, high-level language created by James Gosling for an interactive TV project. It has cross-platform functionality and is consistently among the top of the world's most popular programming languages. Java can be found everywhere, from computers to smartphones to parking meters.

1995: PHP

Formerly known as 'Personal Home Page' which now stands for 'Hypertext Preprocessor', PHP was developed by Rasmus Lerdorf. Its primary uses include building and maintaining dynamic web pages, as well as server-side development. Some of the biggest companies from across the globe use PHP including Facebook, Wikipedia, Digg, WordPress, and Joomla.

2003: Scala

Developed by Martin Odersky, Scala which combines mathematical functional programming and organized object-oriented programming. Scala's compatibility with Java makes it helpful with Android development. LinkedIn, Twitter, Foursquare, and Netflix are just a few examples of the many companies that use Scala in their tech stacks.

2003: Groovy

Derived from Java, Groovy was developed by James Strachan and Bob McWhirter. The language improves productivity because of its succinct and easy to learn. Some well-known companies that are using Groovy in their tech stacks are Starbucks, Transferwise, and Craftbase.

2009: Go

Go was developed by Google to address issues that occur due to large software systems. Due to its simple and modern structure, Go has gained popularity among some of the largest tech companies around the world such as Google, Uber, Twitch, and Dropbox.

2014: Swift

Developed by Apple as a replacement for C, C++, and Objective-C, Swift was developed with the intention to be easier than the aforementioned languages and allow less room for error. Swift's versatility means it can be used for desktop, mobile, and cloud applications. Leading language app 'Duolingo' launched a new app that was written in Swift.

FORTRAN

The initial version of FORTRAN was proposed at IBM by John Backus way back in 1953. It was the first optimizer compiler and was widely used for scientific work because its optimized code competed favorably with hand-written assembly code. The language is still used to this day on some of the world's fastest supercomputers. There has been a string of updates since 1953 with the most recent version released in 2018.

COBOL

COBOL was developed back in 1959 and was one of the first high-level programming languages. Despite being invented so long ago COBOL is still hugely prevalent today. In the US alone, 43% of banking systems are built on COBOL, accounting for 95% of ATM swipes, and 80% of in-person transactions.

Early Microsoft and Apple Programming Languages

Microsoft and Apple are two of the most fundamental companies in the history of programming languages. BASIC (first released in 1964), was modified by Microsoft founders Bill Gates and Paul Allen in 1975. Their modified version became Altair BASIC, which was the first marketed and distributed product of Microsoft, and led to the creation of the company.

Pascal was the programming language of choice during the early days of Apple. Invented in the early 1970s, Apple developers liked Pascal because of its power and ease of use. However, when Apple acquired the NeXTSTEP code library in 1996, Objective-C (which was licensed under NeXTSTEP) became the popular language of choice for the tech giant.

PROCEDURAL LANGUAGES

Procedural programming languages are languages that organize code into functions or procedures that are executed in a specific order.

Procedural Language is also known as 3GL which means third generation language. It is a type of programming language that follows a procedure; set of commands or guidelines that must be followed for smooth execution of the program. It works on step by step basis. It requires the user to tell not only What to do but also How to do it. Its basic idea is to have a program specify the sequence of steps that implements a particular algorithm. Hence, Procedural languages are based on algorithms.

In this, developers use variables, loops, and functions to create a program performing a specific task which does calculations and displays a desired output. It follows a top-down approach. It is carried out in a fixed sequence with a start and end point. The entire program is divided into functions and its main focus is on functions only. The program's code executes linearly with logical steps. In this, the code is written first and executed with some conditions. The procedure calls to decide the conditions.

There are no data-hiding features in procedural language. It is command-driven language. It is operated by means of commands keyed in by the user or issued by the program. In procedural language, both iterative and recursive calls are used. It works through the state of the machine. It returns only restricted data types and allowed values. It is highly efficient despite the fact the size of the program is quite large and is not suitable for critical applications with complex code. It takes a lot of time but it needs very less memory.

Features of Procedural Programming Language

- **Local Variables:** A local variable is a variable that is limited to a specific part of a program and is announced locally in the procedural language, local on the specific function, as the only place to access it.
- **Global Variables:** A global variable is a variable that is declared outside a function and is able to be accessed by any function of the program. It is also

known as the opposite of the local variable.

- **Modularity:** The practice of splitting a program into modules or functions so that each function is devoted to a certain task is referred to as modularity.
- **Pre-Defined Functions:** Pre-defined functions are the amplification of functions which are already defined by the user and don't need the programmer to define them again. A built-in function is already defined in the system libraries, so it is also called a library function.
- **Parameter Passing:** Parameter passing is a mechanism that allows the transfer of data between different functions of the program. The parameters are passed between the called and the calling function, thus allowing data exchange.

Examples of Procedural Language

- **FORTTRAN:** FORTRAN is a short form of FORMula TRANslation. It is an application in critical engineering calculations and high-performance computing.
- **C:** It is a middle-level language. It is basically designed to write System software and requires advanced programming skills to read and write the desired code.
- **BASIC:** BASIC is the short form of Beginner All Purpose Symbolic Instruction Code. It is a high-level programming language that is simple to use.
- **Pascal:** Pascal is easy to learn. It produces transparent, efficient and reliable programs.
- **ALGOL:** ALGOL stands for Algorithmic Language. It is mostly used for scientific calculations.
- **COBOL:** COBOL stands for Common Business Oriented Language. It is easy to read, write and understand. It is basically designed for Business Applications.
- **Java:** Java is a platform independent language. It generates a machine code from source code with the help of Just in Time compiler.

Advantages of Procedural Language

- **Easy to Understand:** Procedural programming stops the programmer from writing more lines of code and depicts the code in very simple steps that can be easily followed by the programmer to understand it.
- **Reusability:** It paves way for easy code reusability.
- **Modularity:** It divides the programs into modules or functions and these functions use different parts of the memory.
- **Simplification:** It simplifies the algorithm and makes it understandable by

the programmer.

- Top-Down Approach: The program flows in linear direction.
- Versatile: Developers can use Procedural language for most basic programming projects.

Disadvantages of Procedural Language

- One can't solve real world problem.
- Encapsulation, Inheritance, Abstraction etc. can't be performed.
- It is less secure as there is no security of data.
- Its semantics are tough and difficult to understand.
- It returns only restricted data types and allowed values.

OBJECT ORIENTED LANGUAGES

As the name suggests, Object-Oriented Programming or OOPs refers to languages that use objects in programming. Object-oriented programming aims to implement real-world entities like inheritance, hiding, polymorphism, etc. in programming. The main aim of OOP is to bind together the data and the functions that operate on them so that no other part of the code can access this data except that function.

OOPs Concepts:

- Class
- Objects
- Data Abstraction
- Encapsulation
- Inheritance
- Polymorphism
- Dynamic Binding
- Message Passing

1. Class:

A class is a user-defined data type. It consists of data members and member functions, which can be accessed and used by creating an instance of that class. It represents the set of properties or methods that are common to all objects of one type. A class is like a blueprint for an object.

For Example: Consider the Class of Cars. There may be many cars with different names and brands but all of them will share some common properties like all of them will have 4 wheels, Speed Limit, Mileage range, etc. So here, Car is the class, and wheels, speed limits, mileage are their properties.

2. Object:

It is a basic unit of Object-Oriented Programming and represents the real-life

entities. An Object is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated. An object has an identity, state, and behavior. Each object contains data and code to manipulate the data. Objects can interact without having to know details of each other's data or code, it is sufficient to know the type of message accepted and type of response returned by the objects.

For example, "Dog" is a real-life Object, which has some characteristics like color, Breed, Bark, Sleep, and Eats.

Object

3. Data Abstraction:

Data abstraction is one of the most essential and important features of object-oriented programming. Data abstraction refers to providing only essential information about the data to the outside world, hiding the background details or implementation. Consider a real-life example of a man driving a car. The man only knows that pressing the accelerators will increase the speed of the car or applying brakes will stop the car, but he does not know about how on pressing the accelerator the speed is increasing, he does not know about the inner mechanism of the car or the implementation of the accelerator, brakes, etc in the car. This is what abstraction is.

4. Encapsulation:

Encapsulation is defined as the wrapping up of data under a single unit. It is the mechanism that binds together code and the data it manipulates. In Encapsulation, the variables or data of a class are hidden from any other class and can be accessed only through any member function of their class in which they are declared. As in encapsulation, the data in a class is hidden from other classes, so it is also known as data-hiding.

Consider a real-life example of encapsulation, in a company, there are different sections like the accounts section, finance section, sales section, etc. The finance section handles all the financial transactions and keeps records of all the data related to finance. Similarly, the sales section handles all the sales-related activities and keeps records of all the sales. Now there may arise a situation when for some reason an official from the finance section needs all the data about sales in a particular month. In this case, he is not allowed to directly access the data of the sales section. He will first have to contact some other officer in the sales section and then request him to give the particular data. This is what encapsulation is. Here the data of the sales section and the employees that can manipulate them are wrapped under a single name "sales section".

5. Inheritance:

Inheritance is an important pillar of OOP (Object-Oriented Programming). The capability of a class to derive properties and characteristics from another class is

called Inheritance. When we write a class, we inherit properties from other classes. So when we create a class, we do not need to write all the properties and functions again and again, as these can be inherited from another class that possesses it. Inheritance allows the user to reuse the code whenever possible and reduce its redundancy.

6. Polymorphism:

The word polymorphism means having many forms. In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form. For example, a person at the same time can have different characteristics. Like a man at the same time is a father, a husband, an employee. So the same person possesses different behavior in different situations. This is called polymorphism.

7. Dynamic Binding:

In dynamic binding, the code to be executed in response to the function call is decided at runtime. Dynamic binding means that the code associated with a given procedure call is not known until the time of the call at run time. Dynamic Method Binding, one of the main advantages of inheritance is that some derived class D has all the members of its base class B. Once D is not hiding any of the public members of B, then an object of D can represent B in any context where a B could be used. This feature is known as subtype polymorphism.

8. Message Passing:

It is a form of communication used in object-oriented programming as well as parallel programming. Objects communicate with one another by sending and receiving information to each other. A message for an object is a request for execution of a procedure and therefore will invoke a function in the receiving object that generates the desired results. Message passing involves specifying the name of the object, the name of the function, and the information to be sent.

Why do we need object-oriented programming?

- To make the development and maintenance of projects more effortless.
- To provide the feature of data hiding that is good for security concerns.
- We can solve real-world problems if we are using object-oriented programming.
- It ensures code reusability.
- It lets us write generic code: which will work with a range of data, so we don't have to write basic stuff over and over again.

FUNCTIONAL PROGRAMMING LANGUAGES

Functional programming languages are based on mathematical functions and the use of recursion and conditional expressions to perform calculations. They

support concepts like abstraction, encapsulation, inheritance, and polymorphism. Some of the popular functional programming languages include: Lisp, Python, Erlang, Haskell, Clojure, etc. Pure Functional Languages – These types of functional languages support only the functional paradigms.

Concepts of Functional Programming.

Pure functions

Recursion

Referential transparency

Functions are First-Class and can be Higher-Order

Variables are Immutable

Pure Functions

These functions have two main properties. First, they always produce the same output for same arguments irrespective of anything else.

Secondly, they have no side-effects i.e. they do not modify any arguments or local/global variables or input/output streams.

Later property is called immutability. The pure function's only result is the value it returns. They are deterministic.

Programs done using functional programming are easy to debug because pure functions have no side effects or hidden I/O. Pure functions also make it easier to write parallel/concurrent applications. When the code is written in this style, a smart compiler can do many things - it can parallelize the instructions, wait to evaluate results when needing them, and memorize the results since the results never change as long as the input doesn't change.

Example of the Pure Function

```
sum(x, y) // sum is function taking x and y as arguments
```

```
return x + y // sum is returning sum of x and y without changing them
```

Recursion

There are no "for" or "while" loop in functional languages. Iteration in functional languages is implemented through recursion. Recursive functions repeatedly call themselves, until it reaches the base case.

example of the recursive function:

```
fib(n)
```

```
if (n <= 1)
```

```
return 1;
```

```
else
```

```
return fib(n - 1) + fib(n - 2);
```

Referential Transparency

In functional programs variables once defined do not change their value

throughout the program. Functional programs do not have assignment statements. If we have to store some value, we define new variables instead. This eliminates any chances of side effects because any variable can be replaced with its actual value at any point of execution. State of any variable is constant at any instant.

Example:

```
x = x + 1 // this changes the value assigned to the variable x.
```

```
// So the expression is not referentially transparent.
```

Functions are First-Class and can be Higher-Order.

First-class functions are treated as first-class variable. The first class variables can be passed to functions as parameter, can be returned from functions or stored in data structures. Higher order functions are the functions that take other functions as arguments and they can also return functions.

Example:

```
show_output(f) // function show_output is declared taking argument f
```

```
// which are another function
```

```
f(); // calling passed function
```

```
print_gfg() // declaring another function
```

```
print("hello gfg");
```

```
show_output(print_gfg) // passing function in another function
```

Variables are Immutable

In functional programming, we can't modify a variable after it's been initialized. We can create new variables – but we can't modify existing variables, and this really helps to maintain state throughout the runtime of a program. Once we create a variable and set its value, we can have full confidence knowing that the value of that variable will never change.

Advantages of Functional Programming

- Pure functions are easier to understand because they don't change any states and depend only on the input given to them. Whatever output they produce is the return value they give. Their function signature gives all the information about them i.e. their return type and their arguments.
- The ability of functional programming languages to treat functions as values and pass them to functions as parameters make the code more readable and easily understandable.
- Testing and debugging is easier. Since pure functions take only arguments and produce output, they don't produce any changes don't take input or produce some hidden output. They use immutable values, so it becomes easier to check some problems in programs written uses pure functions.

- It is used to implement concurrency/parallelism because pure functions don't change variables or any other data outside of it.
- It adopts lazy evaluation which avoids repeated evaluation because the value is evaluated and stored only when it is needed.

Disadvantages of Functional Programming

- Sometimes writing pure functions can reduce the readability of code.
- Writing programs in recursive style instead of using loops can be bit intimidating.
- Writing pure functions are easy but combining them with the rest of the application and I/O operations is a difficult task.
- Immutable values and recursion can lead to decrease in performance.

Applications

- It is used in mathematical computations.
- It is needed where concurrency or parallelism is required.

Why Functional Programming Matters.

Functional programming matters because it offers several benefits that align well with modern computing needs:

- **Immutability and State Management:** In using immutable variables, functional programming result in less or no side effects and it also simpler in thinking through the program. There is also the issue of immutability that assists in managing state efficiently in states like concurrent and parallel computing.
- **Concurrency and Parallelism:** Functions without side effects are naturally concurrency safe because they do not work with shared variables. This makes functional programming most appropriate in the development of programs that support many concurrency and parallelism requirements.
- **Code Readability and Maintainability:** Haskell functions are supposed to be small and deterministic and this characteristic is shared by functions in most functional programming languages. Again, this makes code easier to create, test and debug because it separates the different views of the same functionality. Higher-order functions, not making side effects play a role in the logic of a function, produce code that is more orthogonal.
- **Lazy Evaluation:** Many function programming languages contain code that is lazily evaluated; that is, functions are not actually evaluated until the point at which their values are required. This may result to performance enhancements and possible preclusion of avoidable computations.

Example of functional programming:

Functional programming is supported by several languages, each with unique

features:

- Haskell: Functional language used for purely functional programming language with feature of strong type and lazy evaluation of expressions.
- JavaScript: Is also compatible with functional programming as well as with the others, thanks to which functions may be stored in variables and passed as parameters.
- Python: Comes with such functional programming constructs as higher order function and list comprehension in addition to imperative and object oriented.
- Scala: Supports both Functional and Object-oriented programming paradigms and offers good support for considerations of programming in a functional paradigm.
- Erlang: It is used for concurrency and distributed systems based on functional programming way to process a great number of concurrent processes.
- Lisp: It is one of the first functional languages with symbolical expression processing and rather free-formed macro system.
- Clojure: It is a dialect of Lisp which was originally developed in the late 1950s and early 1960s and it accentuates on functional programming and immutability.

DECLARATIVE PROGRAMMING LANGUAGES.

Declarative programming languages include SQL, HTML, Prolog, Erlang, Haskell, and Lisp. They are used to describe what you want a program to do, rather than how to do it.

Declarative programming is a high-level programming concept, which is the opposite of imperative programming. It is a programming approach in which coders write code that describes what they want to do and what results they want to achieve, rather than how they will achieve the results. It is the job of the program's compiler to figure out the how.

Declarative models rely on preconfigured capabilities in the language without explicit case-by-case instructions on what steps to take to accomplish a particular task. Declarative programming is typically found in databases and configuration management software, paired with a domain-specific language (DSL).

Features of declarative programming languages

High-level syntax

Declarative languages use a clear, concise syntax that's easier to understand than imperative languages.

Abstract

Declarative languages abstract away many of the details of how the program executes.

Focus on results

Declarative languages focus on what result you want to obtain, rather than how you want to get there.

Examples of declarative programming languages

SQL

A declarative language used in data science

HTML

A declarative language used in web development to declare the structure and styling of web pages

Prolog

A declarative language used to build rule-based systems, such as for AI

Haskell

A functional language used for research

Lisp

A functional language that's been used for over 60 years

Declarative programming languages can be divided into two paradigms: functional programming languages and logic programming languages.

Benefits of declarative programming

Declarative programs are easier to work with since they only express computation logic without complicating the code with control flows. Such programs also have fewer unclear procedures, implicit dependencies and mutable states, which also simplifies programming.

And declarative programming minimizes mutability. Immutable objects in code improve its readability and make it easy to understand, even without a whole lot of comments. Immutable code is also easier to update, test and debug, and manage, and it remains safe and valid since background processes cannot cause changes without the programmer's knowledge. Finally, immutable code is traceable, so programmers can keep track of what data is changing and in which parts of the application, and accordingly reload only those parts with each change.

Declarative programming also minimizes state side effects, a situation where a function changes something that's outside its scope, such as changing the value of a variable or generating a nondeterministic output.

Types of declarative programming

The three main types of declarative programming are:

Constraint programming

Constraint programming, also known as constraint optimization, is a programming paradigm in which constraints are declaratively stated for a set of decision variables. The arbitrary constraints help with modeling the problem to be solved without specifying the steps to be executed.

Logic programming

Logic programming involves the use of logical sentences that express certain rules and facts as well as logical inferences based on available data. These languages use queries to display relevant data, a base of existing logic (predicates), and formulas that state facts (atoms).

Constraint logic programming

Constraint logic programs include constraints in a body of clauses. An interpreter is used for execution. It recursively scans the clauses (with its constraints placed in a constraint store) to prove a specific goal.

SCRIPTING LANGUAGES

A scripting language is a type of programming language that's interpreted rather than compiled. This means that the code is translated into machine code when it's run, rather than before.

They do not require the compilation step and are rather interpreted. It brings new functions to applications and glue complex system together. A scripting language is a programming language designed for integrating and communicating with other programming languages.

There are many scripting languages some of them are discussed below:

- **bash:** It is a scripting language to work in the Linux interface. It is a lot easier to use bash to create scripts than other programming languages. It describes the tools to use and code in the command line and create useful reusable scripts and conserve documentation for other people to work with.
- **Node js:** It is a framework to write network applications using JavaScript. Corporate users of Node.js include IBM, LinkedIn, Microsoft, Netflix, PayPal, Yahoo for real-time web applications.
- **Ruby:** There are a lot of reasons to learn Ruby programming language. Ruby's flexibility has allowed developers to create innovative software. It is a scripting language which is great for web development.
- **Python:** It is easy, free and open source. It supports procedure-oriented programming and object-oriented programming. Python is an interpreted language with dynamic semantics and huge lines of code are scripted and is currently the most hyped language among developers.

- Perl: A scripting language with innovative features to make it different and popular. Found on all windows and Linux servers. It helps in text manipulation tasks. High traffic websites that use Perl extensively include priceline.com, IMDB. JavaScript: Used to create interactive web pages, including maps, graphics, and video jukeboxes.
- PHP: An open-source language used for backend web development.

Advantages of scripting languages:

- Easy learning: The user can learn to code in scripting languages quickly, not much knowledge of web technology is required.
- Fast editing: It is highly efficient with the limited number of data structures and variables to use.
- Interactivity: It helps in adding visualization interfaces and combinations in web pages. Modern web pages demand the use of scripting languages. To create enhanced web pages, fascinated visual description which includes background and foreground colors and so on.
- Functionality: There are different libraries which are part of different scripting languages. They help in creating new applications in web browsers and are different from normal programming languages.

Application of Scripting Languages:

Scripting languages are used in many areas:

- Scripting languages are used in web applications. It is used in server side as well as client side. Server side scripting languages are: JavaScript, PHP, Perl etc. and client side scripting languages are: JavaScript, AJAX, jQuery etc.
- Scripting languages are used in system administration. For example: Shell, Perl, Python scripts etc.
- It is used in Games application and Multimedia.
- It is used to create plugins and extensions for existing applications.
- PHP: An open-source language used for backend web development

Characteristics of scripting languages

- They're often used for short scripts rather than full programs
- They're usually short and fast
- They're interpreted from source code or bytecode
- They can be general purpose or domain-specific
- They can be embedded in applications.

Programming methodology

A programming methodology is a set of practices for designing, coding, and testing software. Methodologies can include structured design, object-oriented

design, and more. It encompasses various concepts and techniques that programmers use to develop software applications efficiently and effectively.

The scope of programming ranges from very simple tasks to extremely complicated ones. But behind all the codes, one parameter is common, that is, to handle and solve the problem efficiently. As one can't learn to fly an aeroplane simply by watching it flying, one has to actually learn it. Similarly, to become a proficient coder, one has to actually do the codes. And, the task of coding is extremely easy once one learns how to apply the logic for problem solving. Computer Programming comprises of the fundamental entities which are writing, testing, troubleshooting, debugging, and maintaining of a computer program.

Some of the key concepts of programming methodology include:

- **Algorithms:** An algorithm is a step-by-step procedure for solving a problem or performing a task. Programmers use algorithms to develop the logic for their programs and ensure that they run efficiently and correctly.
- **Data structures:** Data structures are ways of organizing and storing data in a computer. Programmers use data structures to optimize the storage and retrieval of data within their programs.
- **Object-oriented programming (OOP):** OOP is a programming paradigm that focuses on objects, which are instances of classes that encapsulate data and behavior. OOP allows programmers to create modular, reusable, and maintainable code.
- **Functional programming:** Functional programming is a programming paradigm that emphasizes the use of functions to perform computations. Functional programming can improve code clarity, maintainability, and reliability.
- **Software design patterns:** Software design patterns are reusable solutions to common programming problems. They can help programmers write more efficient and maintainable code.
- **Code testing and debugging:** Programmers use testing and debugging techniques to ensure that their code runs correctly and efficiently. This includes techniques such as unit testing, integration testing, and system testing.

Why need Programming Methodology?

Programming methodology is essential for several reasons:

1. **Efficiency:** Programming methodology provides a systematic approach to designing and developing software applications, which can lead to more efficient and effective code. By following best practices and established techniques, programmers can optimize their code for performance, scalability, and maintainability.

2. Reusability: Programming methodology emphasizes the use of modular and reusable code, which can save time and resources in the long run. By developing code that can be easily adapted and reused, programmers can avoid duplicating efforts and create applications more quickly.
3. Quality assurance: Programming methodology includes techniques for testing and debugging code to ensure that it runs correctly and efficiently. By using these techniques, programmers can catch errors and bugs early in the development process, leading to higher-quality software applications.
4. Collaboration: Programming methodology provides a common language and set of tools that programmers can use to collaborate on software projects. By using established practices and techniques, programmers can communicate more effectively and work together more efficiently.
5. Innovation: Programming methodology provides a framework for innovation and experimentation. By using established techniques and best practices as a starting point, programmers can build on existing knowledge and create new solutions to complex problems.

Examples of programming methodologies:

- Waterfall: A classic, sequential method for software development
- Extreme programming: A test-driven methodology that relies on unit testing
- Rapid application development (RAD): A fast methodology that focuses on developing software solutions
- Scrum: An Agile methodology that uses cycles of development called sprints
- DevOps: A methodology that combines development and operations teams to improve efficiency
- Lean: An Agile methodology that aims to reduce waste in software development
- Object-oriented programming (OOPs): A methodology that uses objects to represent real-world entities.
- Dynamic Systems Development Method (DSDM): A methodology for projects with frequent changes in requirements.

SYNTACTIC STRUCTURE.

Syntactic structure refers to the arrangement of words and phrases in a sentence, following grammatical rules and principles. It governs the way components of a sentence are organized and connected, ensuring that the sentence meaning is clear and coherent.

EXPRESSION NOTATION

Expression notation is a way to write mathematical expressions, and includes infix, prefix, and postfix notations. Each notation has its own advantages, depending on whether a human or a computer is using it.

Infix notation

- The most common form of mathematical expression
- The operator is placed between the operands
- For example, $A + B$
- This notation is natural and intuitive for humans because it aligns with traditional arithmetic

Prefix notation

- Also known as Polish notation
- The operator is placed before the operands
- For example, $+ A B$

Postfix notation

- Also known as Reverse Polish Notation (RPN)
- The operator is placed after the operands
- For example, $A B +$

Regular expression notation

- Specifies a pattern of character strings
- Certain characters, called wildcards, have special meanings
- For example, the notation $[0-9]\{3\}$ matches any string of exactly 3 digits

Function notation

- The notation $f(x)$ defines a function named f
- This is read as “ y is a function of x ”

ABSTRACT SYNTAX TREE

An abstract syntax tree (AST) is a tree-like data structure that represents the structure of a program or code snippet. ASTs are a fundamental part of how compilers work.

Abstract Syntax Tree is a kind of tree representation of the abstract syntactic structure of source code written in a programming language. Each node of the tree denotes a construct occurring in the source code.

There is numerous importance of AST with application in compilers as abstract syntax trees are data structures widely used in compilers to represent the structure of program code. An AST is usually the result of the syntax analysis phase of a compiler. It often serves as an intermediate representation of the program through several stages that the compiler requires, and has a strong impact on the final output of the compiler.

IMPORTANCE OF ABSTRACT TREE

AST's are mainly used in compilers to check code for their accuracy. If the generated tree has errors, the compiler prints an error message.

Abstract Syntax Tree (AST) is used because some constructs cannot be represented in context-free grammar, such as implicit typing. They are highly specific to programming languages, but research is underway on universal syntax trees.

LEXICAL SYNTAX

Lexical syntax is a set of rules that defines how a character sequence is broken down into lexeme. A lexeme is an actual string of characters that matches with a pattern and generates a token.

eg- "float", "abs_zero_Kelvin", "=", "-", "273", ";". It's a key part of programming languages, and is often defined in a lexical grammar.

Lexical analysis also known as scanning is the process of converting a sequence of characters in a source code file into a sequence of tokens that can be more easily processed by a compiler or interpreter. It is often the first phase of the compilation process and is followed by syntax analysis and semantic analysis. A lexical token is a sequence of characters that can be treated as a unit in the grammar of the programming languages.

Categories of Tokens

- **Keywords:** In C programming, keywords are reserved words with specific meanings used to define the language's structure like if, else, for, and void. These cannot be used as variable names or identifiers, as doing so causes compilation errors. C programming has a total of 32 keywords.
- **Identifiers:** Identifiers in C are names for variables, functions, arrays, or other user-defined items. They must start with a letter or an underscore (_) and can include letters, digits, and underscores. C is case-sensitive, so uppercase and lowercase letters are different. Identifiers cannot be the same as keywords like if, else or for.
- **Constants:** Constants are fixed values that cannot change during a program's execution, also known as literals. In C, constants include types like integers, floating-point numbers, characters, and strings.
- **Operators:** Operators are symbols in C that perform actions on variables or other data items, called operands.
- **Special Symbols:** Special symbols in C are compiler tokens used for specific purposes, such as separating code elements or defining operations. Examples include; (semicolon) to end statements, (comma) to separate values, {} (curly braces) for code blocks, and [] (square brackets) for arrays. These symbols play a crucial role in the program's structure and syntax.

How does lexical syntax work?

- Lexical syntax rules define the set of possible character sequences (lexemes) of a token.
- A lexer recognizes strings and produces a token for each kind of string.
- Lexical syntax rules omit non-significant portions like whitespace and comments.
- Lexical syntax rules define the basic symbols of a language, such as names, values, and operators.
- During lexical analysis, the source code is scanned character by character and grouped into tokens based on the rules of the programming language. These tokens represent the basic building blocks of the program's syntax, such as keywords, identifiers, punctuation, and constants.
- The lexical analyzer, also known as a lexer or tokenizer, is responsible for performing lexical analysis.
- The output of the lexical analysis phase is a stream of tokens that can be more easily processed by the syntax analyzer, which is responsible for checking the program for correct syntax and structure.
- Lexical analysis is an important step in the compilation process because it ensures that the source code is properly formatted and that the tokens it generates can be easily understood and processed by the compiler or interpreter.

Applications of Lexical Analysis:

Compilers: Lexical analysis is an important part of the compilation process, as it converts the source code of a program into a stream of tokens that can be more easily processed by the compiler.

Interpreters: Lexical analysis is also used in interpreters, which execute a program directly from its source code without the need for compilation.

Text editors: Many text editors use lexical analysis to highlight keywords and other elements of the source code in different colors, making it easier for programmers to read and understand the code.

Code analysis tools: Lexical analysis is used by tools that analyze the source code of a program for errors, security vulnerabilities, and other issues.

Natural language processing: Lexical analysis is also used in natural language processing (NLP) to break down natural language text into individual words and phrases that can be more easily processed by NLP algorithms.

Information retrieval: Lexical analysis is used in information retrieval systems, such as search engines, to index and search for documents based

on the words they contain.

GRAMMAR FOR EXPRESSIONS.

Expression grammar is a type of grammar used in computer science to define the syntax of expressions. It uses formal rules to define the structure of expressions.

Types of expression grammar

Parsing expression grammar: A formal grammar that describes a formal language by defining rules for recognizing strings in the language

Classic expression grammar: A type of grammar that includes unary and binary operators

Basic expression grammar: A type of grammar that uses dollar signs with parentheses to enclose expressions, functions, or field references

How expression grammar works

Expression grammar uses formal rules like concatenation, alternation, and recursion

It allows the formation of expressions with operators like autoincrement, autodecrement, address-of, dereference, boolean complement, and typecasts

It's used to specify and parse, and is a widely-used tool for many string-processing tasks.

Examples of expression grammar

A grammar for URLs can specify the set of sentences that are legal URLs in the HTTP protocol

FORMAL SEMANTICS.

Semantics is the study of linguistic meaning. It examines what meaning is, how words get their meaning, and how the meaning of a complex expression depends on its parts. Part of this process involves the distinction between sense and reference. Sense is given by the ideas and concepts associated with an expression while reference is the object to which an expression point. Semantics contrasts with syntax, which studies the rules that dictate how to create sentences by arranging words, and pragmatics, which investigates how people use language in communication.

In programming language theory, semantics is the rigorous mathematical study of the meaning of programming languages. Semantics assigns computational meaning to valid strings in a programming language syntax. It is closely related to, and often crosses over with, the semantics of mathematical proofs.

Semantics describes the processes a computer follows when executing a program in that specific language. This can be done by describing the relationship between

the input and output of a program, or giving an explanation of how the program will be executed on a certain platform; hence creating a model of computation.

The field of formal semantics encompasses all of the following:

- The definition of semantic models
- The relations between different semantic models
- The relations between different approaches to meaning
- The relation between computation and the underlying mathematical structures from fields such as logic, set theory, model theory, category theory, etc.

It has close links with other areas of computer science such as programming language design, type theory, compilers and interpreters, program verification and model checking.

Approaches to formal semantics:

There are many approaches to formal semantics; these belong to three major classes:

- Denotational semantics: whereby each phrase in the language is interpreted as a denotation, i.e., a conceptual meaning that can be thought of abstractly. Such denotations are often mathematical objects inhabiting a mathematical space, but it is not a requirement that they should be so.

As a practical necessity, denotations are described using some form of mathematical notation, which can in turn be formalized as a denotational metalanguage. For example, denotational semantics of functional languages often translate the language into domain theory. Denotational semantic descriptions can also serve as compositional translations from a programming language into the denotational metalanguage and used as a basis for designing compilers.

- Operational semantics: whereby the execution of the language is described directly (rather than by translation). Operational semantics loosely corresponds to interpretation, although again the "implementation language" of the interpreter is generally a mathematical formalism. Operational semantics may define an abstract machine (such as the SECD machine), and give meaning to phrases by describing the transitions they induce on states of the machine. Alternatively, as with the pure lambda calculus, operational semantics can be defined via syntactic transformations on phrases of the language itself;

- Axiomatic semantics: whereby one gives meaning to phrases by describing the axioms that apply to them. Axiomatic semantics makes no distinction between a phrase's meaning and the logical formulas that describe it; its meaning is exactly what can be proven about it in some logic. The canonical

example of axiomatic semantics is Hoare logic.

Apart from the choice between denotational, operational, or axiomatic approaches, most variations in formal semantic systems arise from the choice of supporting mathematical formalism.

Variations.

Some variations of formal semantics include the following:

- **Action semantics:** Is an approach that tries to modularize denotational semantics, splitting the formalization process in two layers (macro and micro semantics) and predefining three semantic entities (actions, data and yielders) to simplify the specification;
- **Algebraic semantics:** Is a form of axiomatic semantics based on algebraic laws for describing and reasoning about program semantics in a formal manner. It also supports denotational semantics and operational semantics;
- **Attribute Grammars:** Define systems that systematically compute "metadata" (called attributes) for the various cases of the language's syntax. Attribute grammars can be understood as a denotational semantics where the target language is simply the original language enriched with attribute annotations. Aside from formal semantics, attribute grammars have also been used for code generation in compilers, and to augment regular or context-free grammars with context-sensitive conditions;
- **Categorical (or "functorial") semantics:** uses category theory as the core mathematical formalism. A categorical semantics is usually proven to correspond to some axiomatic semantics that gives a syntactic presentation of the categorical structures. Also, denotational semantics are often instances of a general categorical semantics;
- **Concurrency semantics:** Is a catch-all term for any formal semantics that describes concurrent computations. Historically important concurrent formalisms have included the actor model and process calculi;
- **Game semantics:** uses a metaphor inspired by game theory;
- **Predicate transformer semantics:** Developed by Edsger W. Dijkstra, describes the meaning of a program fragment as the function transforming a post condition to the precondition needed to establish it.

For a variety of reasons, one might wish to describe the relationships between different formal semantics. For example:

- To prove that a particular operational semantics for a language satisfies the logical formulas of an axiomatic semantics for that language. Such a proof demonstrates that it is "sound" to reason about a particular (operational) interpretation strategy using a particular (axiomatic) proof system.

- To prove that operational semantics over a high-level machine is related by a simulation with the semantics over a low-level machine, whereby the low-level abstract machine contains more primitive operations than the high-level abstract machine definition of a given language. Such a proof demonstrates that the low-level machine "faithfully implements" the high-level machine.

INFORMAL SEMANTICS.

Informal semantic is a branch of linguistics that deals with the meaning of words and phrases in natural language. Informal semantics focuses on the way people use language in everyday communication, as opposed to formal semantics, which deals with the meaning of words and phrases in a more technical and precise way.

DECLARATION AND DATA TYPES.

In computer programming, a declaration is a language construct specifying identifier properties: it declares a word's (identifier's) meaning. Declarations are most commonly used for functions, variables, constants, and classes, but can also be used for other entities such as enumerations and type definitions. Beyond the name (the identifier itself) and the kind of entity (function, variable, etc.), declarations typically specify the data type (for variables and constants),

In the programming world, Data Type is the set of quantities that belongs together and are of a similar category. Data type is used so that the compiler or interpreter of a programming language can be told about the data which is to be used.

Data Types or Types are attributes that are predefined or can be created by the user so that the program can easily detect the different types of information. This is useful because Computers can understand only Binary Language i.e. 0's and 1's. To perform other basic and complex math operations, you need Data Types that will be understandable in computer language.

Data Types tells the Memory Management Unit (MMU) how much memory is required to store the data before the program compiles.

Each Data Type has a memory size that is pre-defined by the programming language. For example, In Java programming language:

Type Size

Boolean 1 bit

Char 16 bits

Int 32 bits

Float 32 bits

Double 64 bits

Real-life Example of Different Data Types Used

A student login system might require attributes such as Student Name, Age, Birth Date, Roll Number, Address, Class, Division, Time of Login, Mobile Number, etc. Each variable needs to be stored and segregated according to their Data Types during coding to inform the compiler about the expected input data. This will be helpful to classify what type of data does the system contains and it will be easier to work on that.

Different Data Types

Many programming languages have their own pre-defined data types which can be used by simply typing the keywords assigned to them. Few programming languages have their own different Data Types and their characteristics are unique.

Basic Data Types:

1. Int – includes all whole numbers (Eg. 1, 456, 8643)
2. Char – includes all characters (Eg. Letter, Number, Space, Symbol)
3. String – comprises of a set of characters (Eg. Hello Sumit, How are you)
4. Float – contains decimal point numbers (Eg. 9.1, 2.655463, 3.1428571, 1.0)
5. Double – stores fractional numbers (Eg. 2.34, -1.145)
6. Boolean – contains one of 2 possible values only. (Eg. True or False)

Classification of Data Types

Built-in Type/Primitive Type

Data Types that are pre-defined and provided by each Programming Language.

1. Int
2. Char
3. Float
4. Double
5. Boolean

User-Defined Type

These Data Types are defined by the users in the program as per their needs.

1. Structure
2. Array
3. Class
4. Enumeration
5. Union

Composite/Derived Type

These data types are formed by using basic data types i.e. using one or more built-in data types.

1. Array
2. Function
3. Pointer
4. Record
5. Set.

DATA BINDING:

What is data binding?

Data binding is the process that couples two data sources together and synchronizes them. With data binding, a change to an element in a data set automatically updates in the bound data set.

Data binding may be used for many reasons, such as to link an application's user interface (UI) and the data it displays, for data entry, reporting and in text box elements. It also lets internet users manipulate the representation of data in the elements of a web page without needing to use a complicated programming or scripting processes.

Data and data objects of different logic functions can be bound together in data binding. Data types with different languages can also be bound, such as data binding Extensible Markup Language (XML) and UI.

Each data change in one data set automatically reflects in the other bound data set. In binding syntax, the data source is the data provider, and the other data set is the data consumer. The binding forms the link between the data provider and the data consumer, enabling the connection between visual element data and a data source.

Data binding eliminates the need for Document Object Model (DOM) manipulation. DOM is an application programming interface, or API, for Hypertext Markup Language (HTML) and XML.

What are the types of data binding?

Types of data binding are typically defined by their data flow and include the following:

- One-way binding is a relatively simple type of data binding. Changes to the data provider update automatically in the data consumer data set, but not the other way around.
- Two-way binding is where changes to either the data provider or the data consumer automatically updates the other.
- One-way-to-source binding is the reverse of one-way binding. Changes to the data consumer automatically update the data provider but not the other way around.
- One-time binding is where changes to the data provider do not

automatically update the data consumer. This approach is useful when only a snapshot of the data is needed, and the data is static.

Data binding can be simple or complex. Microsoft defines simple data binding as the ability to bind to a single data element. Complex data binding is when multiple elements are bound together.

How to use data binding?

The data binding architecture consists of data source objects (DSOs) that supply information to viewed pages. DSOs also supply information to data consumers that display the DSO information and agents that ensure the data is synchronized between the DSOs and the consumers.

In a Windows .NET framework, both simple and complex data binding options are available. Simple data binding is configured so that one control in the UI is bound to the data value from the data source.

Windows Presentation Foundation in .NET uses data binding by connecting the properties of target objects and data sources, including Common Language Runtime, Language Integrated Query and XML objects. Data templates are also provided to control the presentation of data.

Data binding libraries enable users to bind UI components to data sources in a declarative format. These libraries also provide classes and methods to make changes in data observable. Consequently, collections, fields and objects are more visible.

Data binding examples

The following examples show how data binding can be used:

- Reporting. Binding is a common way to compile reports that display data from a data source to a screen or printer.
- Data entry. Data binding is also a common way to enter large amounts of data and keep it updated and synchronized to a data source.
- Lookup tables. Lookup tables are information tables that are typically a part of larger data displays. Data binding and controls are used to display and change data.
- Master-detail formats. This is a model for communication protocols where one device or process controls another. These formats may have two tables of data bound together.

Data binding tools

Data binding tools include the following:

- Visual Studio is a Microsoft product that provides design tools for working with custom objects as a data source in applications. Visual Studio is also used to bind UI controls. Changes made to objects are automatically made

in a database.

- Data Binding Library is a support library for Android developers that binds UI components to data sources.
- Google Web Toolkit is an open source tool from Google that enables web developers to create and maintain browser-based Java applications deployed as JavaScript. Google Web Toolkit has a feature called UiBinder that allows the creation of UIs.

DATA VISIBILITY.

Data visibility is the ability to access and view data in real-time from different sources. Data visibility helps you to monitor, display, and analyze data for making informed business decisions. Data visibility also allows you to see how data points are affected over time.

Why is data visibility important?

The chief reason data visibility is important for businesses is that it allows organizations to track key metrics and measure the effectiveness of their operations in real-time.

This live insight into their data helps businesses identify areas for improvement and optimize workflows to increase efficiency. Data visibility is also important for improving user experience and streamlining business processes. By providing easy access to data, businesses reduce the time and effort required to complete complex initiatives, make relevant data easier to find and use, and consequently help data users across the business make more efficient, more logical data-driven decisions.

DATA VISIBILITY TOOLS

One of the most important keys to improving data visibility is the implementation of robust data management systems, such as enterprise resource planning (ERP) systems or data loss prevention (DLP) tools. These systems help businesses organize, store, and access their data in a secure and efficient manner. It's also important to establish clear policies and procedures for data access and use, and to ensure that all employees are trained on these protocols.

There are also a number of tools and technologies available that can help businesses improve data visibility. For example, data visualization tools, such as dashboards and charts, can help businesses clearly and effectively communicate data insights to business stakeholders and decision-makers. Additionally, data analytics tools can help businesses mine their data for insights and identify trends and patterns. Some businesses also use automation and automated processes and data classification systems to streamline data management and improve data visibility.

By providing real-time access to data, data visibility provides significant business

value as a function that empowers businesses to improve the efficiency of their operations, reduce waste, enhance the user experience, improve data security, and make intelligent, data-driven decisions.

DATA SCOPE.

Processing large datasets requires a significant amount of forethought; traditional supercomputing models do not provide an ideal match to big data, and so compromises or significantly over-complicated solutions are often necessary to achieve science goals. The Data-Scope endeavors to overcome issues related to big data on traditional HPC by doing the following:

Storing the data local to the compute.

Large datasets are traditionally stored on communal “head” or “storage” nodes, which are shared across hundreds or thousands of nodes. Putting aside IO subsystem obstacles, the data also has to travel across high-speed Infiniband or Ethernet networks to computational nodes that typically have less than a TB of slow, local storage. Thus, the network and the slow disk become bottlenecks.

One-to-One mapping of users to nodes.

Providing users with their own nodes eliminates the problem of sharing large head node spaces with users that require different access patterns and data layouts. Providing exclusive access to a single project (or complementary projects) allows the data to be laid out in a fashion that is conducive for computation.

DATA LIFETIME.

Data lifetime in computing refers to how long a software component keeps data it is processing alive and where components propagate data. The lifetime of a piece of sensitive data can be defined as a set of tuples in which each tuple represents a unique copy of the data. Copies need not be identical, e.g., a password may have a variety of encodings in its lifetime: Unicode, ASCII, UTF-8, keyboard scan codes, base-64 encoded, etc.

OVERVIEW OF TYPE CHECKING.

Type checking is the process of verifying that the types of operands in a program are compatible with the operator being used and that the program is free from type errors before the code is executed.

Type checking is the process of verifying and enforcing constraints of types in values. A compiler must check that the source program should follow the syntactic and semantic conventions of the source language and it should also check the type rules of the language. It allows the programmer to limit what types may be used in certain circumstances and assigns types to values. The type-checker determines whether these values are used appropriately or not.

It checks the type of objects and reports a type error in the case of a violation,

and incorrect types are corrected. Whatever the compiler we use, while it is compiling the program, it has to follow the type rules of the language. Every language has its own set of type rules for the language. We know that the information about data types is maintained and computed by the compiler.

The information about data types like INTEGER, FLOAT, CHARACTER, and all the other data types is maintained and computed by the compiler. The compiler contains modules, where the type checker is a module of a compiler and its task is type checking.

Types of Type Checking:

There are two kinds of type checking:

1. Static Type Checking.
2. Dynamic Type Checking.

Static Type Checking:

Static type checking is defined as type checking performed at compile time. It checks the type variables at compile-time, which means the type of the variable is known at the compile time. It generally examines the program text during the translation of the program. Using the type rules of a system, a compiler can infer from the source text that a function (fun) will be applied to an operand (a) of the right type each time the expression fun(a) is evaluated.

Examples of Static checks include:

- Type-checks: A compiler should report an error if an operator is applied to an incompatible operand. For example, if an array variable and function variable are added together.
- The flow of control checks: Statements that cause the flow of control to leave a construct must have someplace to which to transfer the flow of control. For example, a break statement in C causes control to leave the smallest enclosing while, for, or switch statement, an error occurs if such an enclosing statement does not exist.
- Uniqueness checks: There are situations in which an object must be defined only once. For example, in Pascal an identifier must be declared uniquely, labels in a case statement must be distinct, and else a statement in a scalar type may not be represented.
- Name-related checks: Sometimes the same name may appear two or more times. For example, in Ada, a loop may have a name that appears at the beginning and end of the construct. The compiler must check that the same name is used at both places.

The Benefits of Static Type Checking:

1. Runtime Error Protection.

2. It catches syntactic errors like spurious words or extra punctuation.
3. It catches wrong names like Math and Predefined Naming.
4. Detects incorrect argument types.
5. It catches the wrong number of arguments.
6. It catches wrong return types, like return "70", from a function that's declared to return an int.

Dynamic Type Checking:

Dynamic Type Checking is defined as the type checking being done at run time. In Dynamic Type Checking, types are associated with values, not variables.

Implementations of dynamically type-checked languages runtime objects are generally associated with each other through a type tag, which is a reference to a type containing its type information. Dynamic typing is more flexible. A static type system always restricts what can be conveniently expressed. Dynamic typing results in more compact programs since it is more flexible and does not require types to be spelled out. Programming with a static type system often requires more design and implementation effort.

Languages like Pascal and C have static type checking. Type checking is used to check the correctness of the program before its execution. The main purpose of type-checking is to check the correctness and data type assignments and type-casting of the data types, whether it is syntactically correct or not before their execution.

Static Type-Checking is also used to determine the amount of memory needed to store the variable.

The design of the type-checker depends on:

1. Syntactic Structure of language constructs.
2. The Expressions of languages.
3. The rules for assigning types to constructs (semantic rules).

GARBAGE COLLECTION.

Garbage collection (GC) is a memory recovery feature built into programming languages such as C# and Java. A GC-enabled programming language to include one or more garbage collectors (GC engines) that automatically free up memory space that has been allocated to objects no longer needed by the program.

In other words, garbage collection is the process of removing any objects which are not being used by any other objects. Often abbreviated "GC," garbage collection is a fundamental component of the memory management system used by JavaScript.

Garbage collection ensures that a program does not exceed its memory quota or reach a point that it can no longer function. It also frees up developers from having to manually manage a program's memory, which, in turn, reduces the

potential for memory-related bugs.

In older programming languages, such as C and C++, the developer must manually delete objects and free up memory. Relying on manual processes made it easy to introduce bugs into the code, some of which can have serious consequences.

For example, a developer might forget to free up memory after the program no longer needs it, leading to a memory leak that quickly consumes all the available RAM. Or the developer might free up an object's memory space without modifying a corresponding pointer, resulting in a dangling pointer that causes the application to be buggy or even to crash.

Programming languages that include garbage collection try to eliminate these types of bugs by using carefully designed GC algorithms to control memory deallocation. The garbage collector automatically detects when an object is no longer needed and removes it, freeing up the memory space allocated to that object without affecting objects that are still being used.

Advantages of Garbage collection.

- Frees developers from having to manually release memory.
- Allocates objects on the managed heap efficiently.
- Reclaims objects that are no longer being used, clears their memory, and keeps the memory available for future allocations.

DISADVANTAGES OF GARBAGE COLLECTION.

They consume the computing resources in deciding as to which memory to free and even if the developer knows this.

- The explicit freeing of objects is that garbage collection gives programmers less control over the scheduling of CPU time devoted to reclaiming memory.
- It is impossible to predict when a garbage collector will be invoked and how long it will take to run.
- Garbage collectors stop the entire program while seeking and collecting garbage objects, they can cause arbitrarily long pauses at arbitrary times during the execution of the program.

ABSTRACTION MECHANISM.

Abstraction is an essential feature of Object-Oriented Programming, which implies only keeping the essentials and removing unnecessary information. Abstraction in OOPS is used to hide unnecessary information and display only necessary information to the users interacting.

PROCEDURES

What is procedure abstraction?

Procedural abstraction is the process of breaking down a complex problem into smaller, more manageable steps. By identifying the essential actions that need to

be taken in order to solve a problem, procedural abstraction can help to make the problem-solving process more efficient and effective.

ABSTRACTION FUNCTIONS.

Abstraction functions and representation invariants are internal documentation of a class's implementation details. A client should not need any of this information to properly use the class. This information is recorded to help with implementing, testing, debugging, modifying, and extending the class.

ABSTRACTION ITERATIONS.

Iterators are a generalization of the iteration mechanisms available in most programming languages. They permit users to iterate/repeat over arbitrary types of data in a convenient and efficient way.

REFERENCE VS VALUE

There are two kinds of types in Visual Basic: reference types and value types.

Variables of reference types store references to their data (objects), while variables of value types directly contain their data. With reference types, two variables can reference the same object; therefore, operations on one variable can affect the object referenced by the other variable.

Value Types

A data type is a value type if it holds the data within its own memory allocation.

Value types include the following:

- All numeric data types
- Boolean, Char, and Date
- All structures, even if their members are reference types
- Enumerations, since their underlying type is always SByte, Short, Integer, Long, Byte, UShort, UInteger, or ULong

Every structure is a value type, even if it contains reference type members. For this reason, value types such as Char and Integer are implemented by .NET Framework structures.

You can declare a value type by using the reserved keyword, for example, Decimal. You can also use the New keyword to initialize a value type. This is especially useful if the type has a constructor that takes parameters. An example of this is the Decimal(Int32, Int32, Int32, Boolean, Byte) constructor, which builds a new Decimal value from the supplied parts.

Reference Types

A reference type stores a reference to its data. Reference types include the following:

- String
- All arrays, even if their elements are value types

- Class types, such as Form
- Delegates

A class is a reference type. Note that every array is a reference type, even if its members are value types.

Since every reference type represents an underlying .NET Framework class, you must use the New Operator keyword when you initialize it. The following statement initializes an array.

TYPE PARAMETER

In computer programming languages, Type Parameter is a generic label used in generic programming to reference an unknown data type, data structure, or class. Type Parameter is most frequently used in C++ templates and Java generics.

Generics relates to how classes that store objects can store objects of a freely chosen type. The choice is based on the generic type parameter in the definition of the classes, which makes it possible to choose the type(s) at the moment of the object's creation. Using generics is done in the following manner: after the name of the class, follow it with a chosen number of type parameters. Each of them is placed between the 'smaller than' and 'greater than' signs, like this: `public class Class<TypeParameter1, TypeParameter2, ...>`. The type parameters are usually defined with a single character

Let's implement our own generic class Locker that can hold one object of any type.

```
public class Locker<T> {
    private T element;
    public void setValue(T element) {
        this.element = element;
    }
    public T getValue() {
        return element;
    }
}
```

Depending on the type of an automation, certain parameter limitations might apply. Supported parameter types are string, integer, Boolean, and array.

String

When defining a parameter of type string, an operator running your runbook has a free text input element to enter a parameter value. You can specify the minimum or maximum number of characters that the operator can use. You can also mark a string parameter as a password so that any input that is entered by the operator is not displayed.

Any runbook or automation parameter that is created before June 2021 is of type string.

Integer

When defining a parameter of type integer, an operator running your runbook has an integer input element to enter a parameter value.

You can specify the minimum or maximum value for the integer value.

Boolean

When defining a parameter of type Boolean, an operator running your runbook has a toggle element to enter a parameter value.

Array

When defining a parameter of type array, an operator running your runbook has an input element to define a various number of inputs. The array parameter must define what type of inputs are allowed. You can use an array of either strings, integers, or Booleans.

Enumerations

To restrict the input for a specific parameter to a discrete set of values, you can extend the parameters with an optional enumeration list. Enumeration support is available for parameters of the following types:

- string
- integer
- arrays with string or integer as their item type

You can define any number of allowed discrete values for your enumeration. The user will then be given the choice of selecting any one of them in the Runbook Automation UI, or multiples of those in the case of an array parameter. If a user tries to send a value which is not part of the enumeration, for example via an API call, the action is rejected with an error message.

Additionally, you can assign labels for any or all of the values of an enumeration for integer or string parameters. Labels are optional. If a label is present, the label is displayed in the Runbook Automation UI together with the value.

Example: If you are writing an application which accepts the days of the week as numeric values from 0-6, you can define the following enumeration list with labels:

ValueLabel

0 Monday

1 Tuesday

2 Wednesday

3 Thursday

4 Friday

5 Saturday

6 Sunday

Table 1. Enumeration list with labels

PARAMETERIZED TYPES

A parameterized datatype is a recipe for creating a family of related datatypes. The type variable 'a' is a type parameter for which any other type may be supplied. For example, `int List` is a list of integers, `real List` is a list of reals, and so on.

OBJECT ORIENTED PROGRAMMING PARADIGM

Object-oriented programming (OOP) is defined as a programming paradigm (and not a specific language) built on the concept of objects.

As the name suggests, Object-Oriented Programming or OOPs refers to languages that use objects in programming. Object-oriented programming aims to implement real-world entities like inheritance, hiding, polymorphism, etc in programming. The main aim of OOP is to bind together the data and the functions that operate on them so that no other part of the code can access this data except that function.

OOPs Concepts:

- Class
- Objects
- Data Abstraction
- Encapsulation
- Inheritance
- Polymorphism
- Dynamic Binding
- Message Passing

1. Class:

A class is a user-defined data type. It consists of data members and member functions, which can be accessed and used by creating an instance of that class. It represents the set of properties or methods that are common to all objects of one type. A class is like a blueprint for an object.

For Example: Consider the Class of Cars. There may be many cars with different names and brands but all of them will share some common properties like all of them will have 4 wheels, Speed Limit, Mileage range, etc. So here, Car is the class, and wheels, speed limits, mileage are their properties.

2. Object:

It is a basic unit of Object-Oriented Programming and represents the real-life entities. An Object is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated. An object has an identity, state, and behavior. Each object contains data and code to manipulate the data. Objects can interact without having to

know details of each other's data or code, it is sufficient to know the type of message accepted and type of response returned by the objects.

For example "Dog" is a real-life Object, which has some characteristics like color, Breed, Bark, Sleep, and Eats.

Object

3. Data Abstraction:

Data abstraction is one of the most essential and important features of object-oriented programming. Data abstraction refers to providing only essential information about the data to the outside world, hiding the background details or implementation. Consider a real-life example of a man driving a car. The man only knows that pressing the accelerators will increase the speed of the car or applying brakes will stop the car, but he does not know about how on pressing the accelerator the speed is increasing, he does not know about the inner mechanism of the car or the implementation of the accelerator, brakes, etc in the car. This is what abstraction is.

4. Encapsulation:

Encapsulation is defined as the wrapping up of data under a single unit. It is the mechanism that binds together code and the data it manipulates. In Encapsulation, the variables or data of a class are hidden from any other class and can be accessed only through any member function of their class in which they are declared. As in encapsulation, the data in a class is hidden from other classes, so it is also known as data-hiding.

Consider a real-life example of encapsulation, in a company, there are different sections like the accounts section, finance section, sales section, etc. The finance section handles all the financial transactions and keeps records of all the data related to finance. Similarly, the sales section handles all the sales-related activities and keeps records of all the sales. Now there may arise a situation when for some reason an official from the finance section needs all the data about sales in a particular month. In this case, he is not allowed to directly access the data of the sales section. He will first have to contact some other officer in the sales section and then request him to give the particular data. This is what encapsulation is. Here the data of the sales section and the employees that can manipulate them are wrapped under a single name "sales section".

5. Inheritance:

Inheritance is an important pillar of OOP(Object-Oriented Programming). The capability of a class to derive properties and characteristics from another class is called Inheritance. When we write a class, we inherit properties from other classes. So when we create a class, we do not need to write all the properties and functions again and again, as these can be inherited from another class that possesses it. Inheritance allows the user to reuse the code whenever possible and

reduce its redundancy.

6. Polymorphism:

The word polymorphism means having many forms. In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form. For example, A person at the same time can have different characteristics. Like a man at the same time is a father, a husband, an employee. So the same person possesses different behavior in different situations. This is called polymorphism.

7. Dynamic Binding:

In dynamic binding, the code to be executed in response to the function call is decided at runtime. Dynamic binding means that the code associated with a given procedure call is not known until the time of the call at run time. Dynamic Method Binding One of the main advantages of inheritance is that some derived class D has all the members of its base class B. Once D is not hiding any of the public members of B, then an object of D can represent B in any context where a B could be used. This feature is known as subtype polymorphism.